AcciSure

Accidental Insurance Management Platform

Documentation Report Complete Technical & Functional Reference

Table of Contents

1	**Introduction**	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions and Acronyms	4
1.4	Document Conventions	4
2	**Overall Description**	5
2.1	Product Perspective	5
2.2	Product Functions Overview	5
2.3	User Characteristics	6
2.4	Constraints	7
2.5	Assumptions and Dependencies	7
3	**System Architecture**	8
3.1	Architecture Overview	8
3.2	Data Flow Architecture	9
3.3	Security Architecture	9
4	**Database Schema**	10
4.1	Entity Overview	10
4.2	Core Tables	10
5	**Specific Requirements**	16
5.1	Functional Requirements — Authentication	16
5.2	Functional Requirements — Policy Management	17
5.3	Functional Requirements — Claims Management	19
5.4	Functional Requirements — Communication	21
5.5	Functional Requirements — Administration	22
5.6	Functional Requirements — AI Integrations	23
6	**API Reference**	24
6.1	AuthController	24
6.2	AdminController	24
6.3	AgentController	25
6.4	PolicyController	26
6.5	ClaimController	27
6.6	ChatController	28
6.7	NotificationsController	28
6.8	PaymentController	29

6.9	ReportController	29
6.10	DashboardController	29
7	**Non-Functional Requirements**	30
7.1	Performance Requirements	30
7.2	Security Requirements	30
7.3	Availability & Reliability	31
7.4	Usability Requirements	31
7.5	Maintainability	31
8	**External Interface Requirements**	32
8.1	User Interface Requirements	32
8.2	Software Interface Requirements	32
8.3	Hardware Interface Requirements	33
9	**Innovations & USPs**	34
9.1	Implemented Innovations	34
9.2	Upcoming Features	37
9.3	Competitive Differentiation	37
10	**Workflow Summaries**	38
10.1	Customer Workflow	38
10.2	Agent Workflow	39
10.3	Claims Officer Workflow	39
10.4	Admin Workflow	40
11	**Appendices**	41

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive and formal description of the AcciSure Accidental Insurance Management Platform. It defines the functional and non-functional requirements, system architecture, database design, API specifications, innovation features, and role-based workflows that govern the entire platform.

This document is intended to serve as the definitive technical and functional reference for:

- Software developers and engineers building or extending the platform
- Quality assurance engineers designing test cases and acceptance criteria
- Product managers and business analysts evaluating feature completeness
- Stakeholders, investors, and evaluators assessing the platform's technical maturity
- System administrators responsible for deployment, security, and maintenance

1.2 Scope

AcciSure is a full-stack, AI-powered, role-based digital accidental insurance management platform that digitalises the complete lifecycle of accidental insurance — spanning customer registration, policy discovery, premium calculation, policy application, agent review, claims filing, AI-assisted claims analysis, settlement, and real-time communication.

The platform scope includes the following major functional domains:

- Customer-facing portal: Registration, policy browsing, AI chatbot, premium calculation, application, payment, claims, and chat
- Agent portal: Application review, customer management, real-time chat, video calls, and commission tracking
- Claims Officer portal: Claims queue management, AI-assisted document analysis, and settlement decisions
- Admin portal: User management, workload-based assignment, KPI dashboards, and financial reporting
- AI integrations: N8N workflow engine for policy chatbot and claims intelligence
- Real-time communications: SignalR WebSocket chat and WebRTC video calling
- Cloud infrastructure: ImageKit CDN for document storage, JWT-based authentication

Out of scope for v1.0.0:

- Mobile native applications (iOS/Android) — web-responsive design only
- Integration with external insurance regulatory portals (IRDAI APIs)
- Multi-currency or international payment gateways
- AI Voice Call Assistant — planned for v2.0.0
- Multilingual support — planned for v2.0.0

1.3 Definitions and Acronyms

Term / Acronym	Definition
SRS	Software Requirements Specification — this document
API	Application Programming Interface — RESTful HTTP endpoints exposed by the backend
JWT	JSON Web Token — stateless authentication token used across all secured endpoints
OTP	One-Time Password — time-limited numeric code sent via SMS for phone verification
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
N8N	An open-source, self-hostable workflow automation engine used for AI integrations
WebRTC	Web Real-Time Communication — browser-native peer-to-peer video/audio protocol
SignalR	Microsoft ASP.NET Core library for real-time WebSocket-based communication
CDN	Content Delivery Network — geographically distributed servers for fast content delivery
EF Core	Entity Framework Core — ORM (Object-Relational Mapper) for .NET
INR	Indian Rupee — the currency used throughout all monetary fields
FR	Functional Requirement
NFR	Non-Functional Requirement
KYC	Know Your Customer — identity verification using Aadhaar number
SPA	Single Page Application — frontend architecture that loads a single HTML page
STUN/TURN	Session Traversal Utilities for NAT / Traversal Using Relays around NAT — WebRTC server types

1.4 Document Conventions

Throughout this SRS, the following conventions apply:

- Requirement identifiers follow the format FR_[DOMAIN]_[NUMBER] for functional requirements and NFR_[DOMAIN]_[NUMBER] for non-functional requirements
- All monetary values are expressed in Indian Rupees (INR)
- All datetime values follow ISO 8601 UTC format
- API endpoints are prefixed with /api/ and described with their HTTP method
- Database column names are presented in PascalCase matching the SQL schema
- Bold text in tables indicates primary or key attributes

2. Overall Description

2.1 Product Perspective

AcciSure is a standalone, end-to-end digital insurance platform designed as a complete replacement for traditional paper-based and semi-digital accidental insurance workflows. It does not integrate into an existing legacy insurance system but instead provides a fresh, modern architecture built from the ground up on contemporary web technologies.

The platform sits at the intersection of enterprise SaaS software and AI-augmented decision-making. It connects four distinct user types through a unified backend API, a relational SQL Server database, and a real-time communication layer. The system interfaces with several external services:

- N8N Workflow Automation Engine — for AI chatbot and claims intelligence
- ImageKit.io CDN — for cloud document storage and delivery
- Twilio SMS Gateway — for OTP verification during registration
- Google reCAPTCHA — for bot-prevention on public registration endpoints
- MailKit / SendGrid — for automated email dispatch of staff credentials
- STUN/TURN servers — for WebRTC video call infrastructure

2.2 Product Functions Overview

At the highest level, AcciSure performs the following major functions:

Functional Area	Description
Customer Onboarding	Dual-verified registration (OTP + CAPTCHA), JWT-based login, role-gated dashboard access
Policy Discovery	Public browsing of Individual and Family Accidental Insurance plans across Silver, Gold, and Premium tiers
AI Policy Guidance	Contextual N8N-powered chatbot answers policy-specific questions inline before purchase commitment
Premium Calculation	Dynamic premium quotes based on age, profession, alcohol/smoking habits, and travel frequency with risk multipliers
Policy Application	Formal application submission, agent assignment by Admin, agent review and approval, online payment activation
Real-Time Communication	WhatsApp-style SignalR chat and WebRTC video calling between customers and their assigned agents
Claims Filing	Multipart claim submission with up to 10 document uploads, incident details, and geographic location
AI Claims Intelligence	Automated AI analysis of claim documents producing risk scores, summaries, and suggested payout ranges
Claims Settlement	Officer approval/rejection with automated coverage balance deduction and full audit trail
Admin Oversight	Platform-wide KPI dashboards, load-based staff assignment, user management, and financial reporting

2.3 User Characteristics

AcciSure operates with four distinct user roles, each with clearly defined responsibilities, access privileges, and technical proficiency expectations:

2.3.1 Customer

Customers are the primary consumers of AcciSure's services. They range from young working professionals (18–30) to mid-career individuals (31–60) seeking individual or family accidental insurance coverage. Expected technical proficiency is basic — ability to use a web browser and fill online forms. Customers interact with the platform through guided wizards, AI assistance, and intuitive dashboards. They do not require any insurance domain knowledge as the AI chatbot and premium calculator guide their journey.

2.3.2 Agent

Agents are internal insurance sales brokers and relationship managers employed by the insurer. They have moderate technical proficiency — comfortable using web-based CRM tools. Agents receive applications assigned by the Admin, review applicant data, make approval/rejection decisions, and maintain customer relationships via chat and video. They track commission performance through analytics dashboards.

2.3.3 Claims Officer

Claims Officers are specialist adjudicators responsible for evaluating insurance claims. They possess insurance domain expertise and moderate-to-high technical proficiency. Their primary workflow involves reviewing AI-generated claim analysis alongside original documents, then making final approval/rejection decisions with specified payout amounts. They are assigned claims by the Admin based on current workload.

2.3.4 Admin

Admins have full system access and are responsible for platform governance. They require high technical and operational proficiency — understanding of user management, workload distribution, and financial oversight. Admins create and provision agent and claims officer accounts, assign work based on real-time load metrics, and monitor platform-wide KPIs through the admin dashboard.

2.4 Constraints

The following constraints govern the design and implementation of AcciSure v1.0.0:

- Regulatory: All customer data handling must comply with applicable Indian data protection regulations. Aadhaar numbers are collected for KYC compliance purposes only.
- Technical — File Uploads: Claim documents are restricted to PDF, JPG, JPEG, and PNG formats. Maximum file size is 5 MB per file, with a maximum of 10 files per claim.
- Technical — Age Limits: Policy applications are restricted to applicants aged 18 to 60, enforced at both the API validation and frontend form level.
- Technical — Password Policy: Minimum 8 characters, must include uppercase, lowercase, at least one digit, and one special character.
- Technical — Rate Limiting: All API endpoints are rate-limited to 100 requests per minute per user/IP to prevent abuse.

- Browser Support: The frontend must function correctly on the latest two major versions of Chrome, Firefox, and Edge. Internet Explorer is explicitly not supported.
- Currency: All monetary values are denominated in Indian Rupees (INR). No multi-currency support is provided in v1.0.0.
- Connectivity: WebRTC video calling and SignalR chat require stable internet connectivity. Degraded performance under poor connectivity is expected and documented.

2.5 Assumptions and Dependencies

The following assumptions underpin the v1.0.0 design:

- N8N is self-hosted within the same infrastructure network as the AcciSure backend API, ensuring low-latency webhook communication.
- ImageKit.io remains available as a cloud service. Document URLs stored in the database are assumed to remain permanently accessible via ImageKit CDN.
- Twilio (or equivalent SMS gateway) is operational for OTP dispatch. OTP delivery failures are handled gracefully with a resend mechanism on the frontend.
- All platform users have access to devices with a modern browser supporting WebRTC (camera/microphone access) for video calling features.
- SQL Server is deployed with sufficient storage to handle the projected volume of user records, policy applications, claims, and chat messages.
- Email delivery via MailKit/SendGrid is reliable for credential dispatch. Bounce handling is out of scope for v1.0.0.
- The STUN/TURN server infrastructure (via Agora SDK or equivalent) is available for WebRTC session establishment.

3. System Architecture

3.1 Architecture Overview

AcciSure follows a clean three-tier architecture separating the presentation layer, business logic, and data persistence. The architecture is designed for horizontal scalability, stateless API operation, and modular expansion of features.

Layer	Technology	Responsibility
Presentation Layer	Angular 21 / React.js SPA	Role-based routing, real-time chat UI, policy browsing, AI chatbot widget, video calling, forms, notifications
API Layer	ASP.NET Core 8 Web API	44 RESTful endpoints across 10 controllers, JWT auth, role authorization, SignalR hub, business logic
Data Layer	SQL Server + EF Core 8	13-table relational schema, foreign key integrity, indexed queries, LINQ-based data access
Cloud Storage	ImageKit.io CDN	Claim document upload, CDN delivery, secure URL generation, file metadata stored in ClaimDocuments table
AI / Automation	N8N Workflow Engine	Policy chatbot AI responses, claims document analysis producing summaries, risk scores, and payout ranges
Email Automation	MailKit / SendGrid	Automated credential dispatch to newly created agents and claims officers on account creation
Auth & Security	JWT + ASP.NET Identity	Token-based stateless auth, bcrypt password hashing, role-based access control, OTP + CAPTCHA at registration
Real-Time Comms	SignalR + WebRTC	Zero-polling bidirectional chat messaging; peer-to-peer video/audio calls via STUN/TURN servers

3.2 Data Flow Architecture

The standard request lifecycle in AcciSure follows this flow:

1. The client (browser SPA) sends an HTTP request with a JWT Bearer token in the Authorization header.
2. The ASP.NET Core middleware pipeline validates the JWT token, extracts user identity and role claims.
3. The appropriate controller method is invoked. Role-based [Authorize] attributes reject unauthorized role access with HTTP 403.
4. The controller invokes the service layer, which uses Entity Framework Core to build parameterized SQL queries.
5. EF Core executes the queries against SQL Server and maps results to strongly-typed C# model objects.
6. For file operations, the service calls the ImageKit SDK to upload documents and receive CDN URLs.

7. For AI features, the service posts a webhook to the N8N engine and awaits the structured JSON response.
8. For real-time notifications, the SignalR hub broadcasts messages to all connected clients in the target group.
9. The controller serializes the response to JSON and returns it to the client with the appropriate HTTP status code.

3.3 Security Architecture

AcciSure implements defence-in-depth security across all layers:

- Authentication: JWT Bearer tokens are issued on successful login, validated cryptographically on every protected request. Tokens carry user ID, email, and role claims.
- Password Security: ASP.NET Identity handles bcrypt password hashing with automatic salting. Plain-text passwords are never stored.
- Registration Security: Google reCAPTCHA prevents automated bot registrations at the form submission level. SMS OTP verifies phone number ownership before account creation.
- Role-Based Authorization: Every sensitive API endpoint is decorated with [Authorize(Roles = "...")] attributes enforcing strict role boundaries. Cross-role access returns HTTP 403.
- Rate Limiting: 100 requests/minute/user enforced at the API gateway level prevents abuse and DoS attempts.
- File Validation: Only PDF, JPG, JPEG, and PNG are accepted for claim document uploads. File size is capped at 5 MB per file with a maximum of 10 files per claim to prevent storage abuse.
- CORS: Configured allowed origins restrict browser-based API access to trusted frontend domains only.
- WebRTC Security: All video streams use DTLS (Datagram Transport Layer Security) encryption as mandated by the WebRTC specification.
- Data Isolation: Each role can only access data within their permitted scope — agents see only their assigned customers, officers see only their assigned claims.

4. Database Schema

4.1 Entity Overview

The AcciSure platform uses Microsoft SQL Server 2019+ as its primary relational database. The schema is managed through EF Core migrations and consists of 13 tables: 7 standard ASP.NET Identity tables (extended with custom fields) and 6 business-logic tables. All tables use appropriate indexing, foreign key constraints, and cascade rules.

Table Name	Type	Purpose
AspNetUsers	Identity (Extended)	All platform users: Customers, Agents, Officers, Admins
AspNetRoles	Identity	Role definitions: Admin, Agent, Customer, ClaimOfficer
AspNetUserRoles	Identity	Many-to-many user-role assignments
AspNetUserClaims	Identity	User-level claims for fine-grained authorization
AspNetUserLogins	Identity	External OAuth login provider references
AspNetUserTokens	Identity	Authentication tokens and refresh token storage
AspNetRoleClaims	Identity	Role-level claims for permissions management
Notifications	Business	System-generated notifications for policy/claim status changes
PolicyApplications	Business (Core)	Policy lifecycle tracking from submission to expiry with coverage ledger
InsuranceClaims	Business (Core)	Claim lifecycle tracking from filing to officer decision
ClaimDocuments	Business	Cloud document metadata and ImageKit CDN URLs per claim
Chats	Business	Chat session metadata per policy between customer and agent
ChatMessages	Business	Individual messages within chat sessions with read receipt flags

4.2 Core Tables — Detailed Schema

4.2.1 AspNetUsers (Extended Identity Table)

The central user table storing all platform users. Extended with custom insurance-specific fields beyond the standard ASP.NET Identity columns.

Column	Type	Constraints	Description

Id	NVARCHAR(450)	PK	GUID — unique user identifier
FullName	NVARCHAR(100)	NOT NULL	User's full legal name
AadhaarNumber	NVARCHAR(12)	NULL	12-digit Aadhaar for Customer KYC verification
BankAccountNumber	NVARCHAR(16)	NULL	16-digit bank account for Agent commission payouts
CreatedAt	DATETIME2	DEFAULT UTC	Account creation timestamp in UTC
InitialPassword	NVARCHAR(MAX)	NULL	Stored for Admin-created accounts for credential email dispatch
Email	NVARCHAR(256)	UNIQUE	Primary login credential and communication address
PasswordHash	NVARCHAR(MAX)	NULL	bcrypt-hashed password managed by ASP.NET Identity
PhoneNumber	NVARCHAR(MAX)	NULL	Mobile number with country code for OTP verification
PhoneNumberConfirmed	BIT	DEFAULT 0	Set to 1 only after successful OTP verification
LockoutEnabled	BIT	DEFAULT 0	Enables account lockout after repeated failed logins
AccessFailedCount	INT	DEFAULT 0	Counter for consecutive failed login attempts

4.2.2 PolicyApplications (Core Business Table)

The central business table tracking the complete lifecycle of every insurance policy application from initial submission through agent review, payment activation, coverage tracking, and expiry.

Column	Type	Description
Id	NVARCHAR(450)	Primary key — unique policy application GUID
UserId	NVARCHAR(450)	FK → AspNetUsers — the customer who applied (CASCADE delete)
PolicyCategory	NVARCHAR(MAX)	e.g., 'Health Insurance - Individual', 'Health Insurance - Family'
TierId	NVARCHAR(MAX)	Plan tier: SILVER, GOLD, or PREMIUM
CalculatedPremium	DECIMAL(18,2)	Final calculated annual/periodic premium in INR
Status	NVARCHAR(MAX)	Lifecycle state: Pending → Assigned → Approved → Active → Expired (or Rejected)
ApplicationDataJson	NVARCHAR(MAX)	Full applicant JSON: name, age, profession, habits, nominee, family members
AssignedAgentId	NVARCHAR(450)	FK → AspNetUsers — agent assigned by Admin (NO ACTION on delete)

TotalCoverageAmount	DECIMAL(18,2)	Total sum insured per the purchased plan tier in INR
RemainingCoverageAmount	DECIMAL(18,2)	Coverage remaining after deducting approved claim payouts — auto-updated on approval
TotalApprovedClaimsAmount	DECIMAL(18,2)	Running total of all approved claim amounts for audit trail purposes
PaymentMode	NVARCHAR(MAX)	Premium payment frequency: Annual / Semi-Annual / Quarterly / Monthly
StartDate	DATETIME2	Policy activation date — set on successful payment processing
ExpiryDate	DATETIME2	Policy expiry date — calculated from StartDate + selected policy duration
TransactionId	NVARCHAR(MAX)	Payment transaction reference number for reconciliation

4.2.3 InsuranceClaims

Stores all insurance claims raised by customers against their active policies. Tracks the full claims lifecycle from customer submission through Admin assignment to officer decision.

Column	Type	Description
Id	NVARCHAR(450)	Primary key — unique claim GUID
PolicyApplicationId	NVARCHAR(450)	FK → PolicyApplications — the active policy this claim is against
UserId	NVARCHAR(450)	FK → AspNetUsers — the customer who raised the claim
RequestedAmount	DECIMAL(18,2)	INR amount claimed by the customer based on incident severity
ApprovedAmount	DECIMAL(18,2)	INR amount approved by the Claims Officer (0 if rejected)
IncidentType	NVARCHAR(MAX)	Category: Accident / Sickness / Death / Property Damage
IncidentLocation	NVARCHAR(MAX)	Geographic location string where the incident occurred
IncidentDate	DATETIME2	Date of the incident — must be in the past (validated)
HospitalName	NVARCHAR(MAX)	Name of treating hospital or clinic
HospitalizationRequired	BIT	Whether inpatient hospital admission was required for treatment
Status	NVARCHAR(MAX)	Claim lifecycle state: Pending → Assigned → Approved / Rejected
AssignedClaimOfficerId	NVARCHAR(450)	FK → AspNetUsers — officer assigned by Admin to review the claim

Remarks	NVARCHAR(MAX)	Officer's documented rationale for approval or rejection decision
ProcessedAt	DATETIME2	Exact UTC timestamp when the officer made their final decision

4.2.4 ClaimDocuments

Stores metadata and CDN access URLs for all documents uploaded by customers when raising a claim. Actual files are stored in ImageKit.io cloud storage.

Column	Type	Description
Id	NVARCHAR(450)	Primary key — unique document record GUID
ClaimId	NVARCHAR(450)	FK → InsuranceClaims — the claim this document belongs to (CASCADE delete)
FileId	NVARCHAR(MAX)	ImageKit-assigned unique file identifier for management operations
FileName	NVARCHAR(MAX)	Original filename as uploaded by the customer (e.g., medical_bill.pdf)
FileUrl	NVARCHAR(MAX)	Public CDN URL for direct document access — consumed by Claims Officers and N8N AI
FileSize	BIGINT	File size in bytes — validated against 5 MB maximum at upload
UploadedAt	DATETIME2	UTC timestamp of upload to ImageKit cloud storage

4.2.5 Chats and ChatMessages

The Chats table stores session metadata for each customer-agent conversation, linked to a specific policy. ChatMessages stores individual messages within those sessions.

Chats Column	Type	Description
Id	NVARCHAR(450)	Unique chat session GUID
PolicyId	NVARCHAR(450)	FK → PolicyApplications — the policy this chat session is associated with
CustomerId / AgentId	NVARCHAR(MAX)	User IDs of the conversation participants
LastMessage	NVARCHAR(MAX)	Preview text of the most recent message for inbox list views
LastMessageTime	DATETIME2	Timestamp of the last message for sorting conversations by recency

ChatMessages Column	Type	Description
Id	NVARCHAR(450)	Unique message GUID

ChatId	NVARCHAR(450)	FK → Chats — the session this message belongs to (CASCADE delete)
SenderId	NVARCHAR(MAX)	User ID of the message sender (Customer or Agent)
SenderRole	NVARCHAR(MAX)	Role label: 'Customer' or 'Agent' — for display without JOIN queries
Message	NVARCHAR(MAX)	Full text content of the message
Timestamp	DATETIME2	UTC timestamp of message creation
IsRead	BIT	Read receipt flag — updated via POST /api/chat/{policyId}/read

5. Specific Requirements - Functional

This section defines all functional requirements for the AcciSure platform. Each requirement is identified with a unique ID, a priority level (Critical / High / Medium), and a detailed description of the expected system behaviour.

5.1 Functional Requirements — Authentication & Registration

Req. ID	Priority	Description
FR_AUTH_01	Critical	The system must allow new customers to register by providing: FullName, Email, Password (min 8 chars, mixed case + digit + symbol), PhoneNumber, and optionally AadhaarNumber.
FR_AUTH_02	Critical	Registration must require Google reCAPTCHA completion before the POST /api/auth/register endpoint is called, preventing automated bot registrations.
FR_AUTH_03	Critical	After CAPTCHA validation, the system must dispatch an OTP via SMS to the provided phone number. Account creation is blocked until OTP verification succeeds. PhoneNumberConfirmed is set to true on success.
FR_AUTH_04	Critical	On successful registration, the system must return a JWT Bearer token containing the user ID, email, and assigned role. The token must be used for all subsequent authenticated requests.
FR_AUTH_05	Critical	The system must authenticate registered users via POST /api/auth/login with email and password. On success, return a JWT token, email, roles array, and userId. On failure, return HTTP 401 with a non-specific error message.
FR_AUTH_06	High	The system must reject duplicate email registrations with HTTP 400 and the message 'Email already exists' without revealing whether the email is registered.
FR_AUTH_07	High	All protected endpoints must validate the JWT token on every request. Expired or tampered tokens must return HTTP 401. Missing tokens must return HTTP 401.
FR_AUTH_08	Medium	The system must enforce account lockout after a configurable number of consecutive failed login attempts (AccessFailedCount), returning HTTP 423 when locked.

5.2 Functional Requirements — Policy Management

Req. ID	Priority	Description
FR_POL_01	Critical	The system must expose GET /api/policy/configuration as a public (unauthenticated) endpoint returning all available insurance plans with tier details, base pricing, coverage amounts, and risk multipliers.
FR_POL_02	Critical	Individual Accident Policy tiers: Silver (₹5L coverage, from ₹3,000/yr), Gold (₹10L coverage, from ₹5,000/yr — Most Popular), Premium (₹20L coverage, from ₹8,000/yr). Benefits escalate across tiers.

FR_POL_03	Critical	Family Group Policy tiers: Family Silver (₹10L, from ₹7,000/yr), Family Gold (₹20L, from ₹12,000/yr — Most Popular), Family Premium (₹40L, from ₹18,000/yr). Covers up to 4 members.
FR_POL_04	Critical	The system must provide POST /api/policy/calculate-premium for authenticated users to receive a premium quote without submitting a formal application. Calculation applies risk multipliers based on applicant age (18-30: 1.0x, 31-45: 1.2x, 46-60: 1.5x), profession, smoking habit, alcohol habit, and monthly travel distance.
FR_POL_05	Critical	The system must validate that applicant age is between 18 and 60. Requests with age outside this range must be rejected with HTTP 400 and the message 'Age must be between 18 and 60'.
FR_POL_06	Critical	POST /api/policy/apply must create a PolicyApplication record with status 'Pending' and store the complete applicant data as JSON in ApplicationDataJson. The endpoint returns applicationId, calculatedPremium, and status.
FR_POL_07	High	Admin must be able to view all policy applications via GET /api/admin/policy-requests with full details. Admin must be able to view real-time agent workload via GET /api/admin/agents-with-load showing current assigned policy count per agent.
FR_POL_08	Critical	Admin must be able to assign a policy application to a specific agent via POST /api/admin/assign-agent with applicationId and agentId. Assignment updates application status to 'Assigned' and triggers a notification to the agent.
FR_POL_09	Critical	Assigned agents must be able to review applications via GET /api/agent/my-requests and make approval/rejection decisions via POST /api/agent/review-request. The customer receives a real-time notification of the agent's decision.
FR_POL_10	Critical	Customers with approved applications must be able to process payment via POST /api/payment/process providing applicationId and amount matching calculatedPremium. On success, policy status changes to 'Active', StartDate and ExpiryDate are set, and the agent receives a 10% commission.
FR_POL_11	High	GET /api/policy/my-policies must return all policies for the authenticated customer including: policyCategory, tierId, status, calculatedPremium, totalCoverageAmount, remainingCoverageAmount, startDate, expiryDate, and assignedAgentId.
FR_POL_12	High	The system must automatically calculate and record a 10% commission (of the calculated premium) against the assigned agent's record in the database upon successful policy payment and activation.

5.3 Functional Requirements — Claims Management

Req. ID	Priority	Description
FR_CLM_01	Critical	Customers with active policies must be able to raise a claim via POST /api/claim/raise using multipart/form-data. Required fields: policyApplicationId, incidentType, incidentLocation, incidentDate, description, hospitalName, hospitalizationRequired, requestedAmount.
FR_CLM_02	Critical	The claim submission endpoint must accept document uploads with the following constraints: accepted formats are PDF, JPG, JPEG, and PNG only; maximum file size is 5 MB per file; maximum 10 files per claim. Violations must return HTTP 400.
FR_CLM_03	Critical	Each uploaded document must be sent to ImageKit.io cloud storage via the ImageKit SDK. The returned FileId and FileUrl must be stored in the ClaimDocuments table with FileName, FileSize, and UploadedAt metadata.
FR_CLM_04	High	For Family Group Policies, the claim form must additionally capture AffectedMemberName and AffectedMemberRelation (Spouse / Son / Daughter / Parent) to identify which covered family member the claim pertains to.
FR_CLM_05	Critical	Admin must be able to view all pending (unassigned) claims via GET /api/claim/admin/pending and view real-time officer workload via GET /api/claim/admin/officers. Admin assigns a claims officer via POST /api/claim/admin/assign.
FR_CLM_06	Critical	When a Claims Officer opens an assigned claim, the system must automatically trigger an N8N AI workflow webhook. The webhook retrieves all ClaimDocument FileUrls, analyzes document content, and returns a structured JSON response containing: (1) plain-language document summary, (2) risk factor score (Low/Medium/High), (3) suggested claim amount range in INR, (4) overall risk rating.
FR_CLM_07	Critical	The AI-generated analysis must be displayed alongside the original claim documents and incident details in the Claims Officer's review interface. The officer retains full decision authority — the AI output is advisory only.
FR_CLM_08	Critical	Claims Officers must submit their decision via POST /api/claim/officer/review with: claimId, status ('Approved' or 'Rejected'), approvedAmount (INR; must be 0 if status is 'Rejected'), and remarks documenting the decision rationale.
FR_CLM_09	Critical	Upon claim approval, the system must atomically: (1) update InsuranceClaims.ApprovedAmount and InsuranceClaims.ProcessedAt, (2) deduct the approved amount from PolicyApplications.RemainingCoverageAmount, (3) add the approved amount to PolicyApplications.TotalApprovedClaimsAmount. All three updates must succeed or all must fail (atomic transaction).
FR_CLM_10	High	The system must prevent over-claiming: ApprovedAmount must not exceed the policy's current RemainingCoverageAmount. Requests exceeding the remaining balance must be rejected with HTTP 400.
FR_CLM_11	High	Customers must be able to view all their claims and current status via GET /api/claim/my-claims, including the latest approved amount and officer remarks for processed claims.

FR_CLM_12	High	Agents must be able to view claims raised by their assigned customers via GET /api/claim/agent/customer-claims to maintain full portfolio visibility.

5.4 Functional Requirements — Real-Time Communication

Req. ID	Priority	Description
FR_COM_01	High	The system must provide WhatsApp-style real-time messaging between customers and their assigned agents using ASP.NET Core SignalR WebSocket connections authenticated with JWT Bearer tokens.
FR_COM_02	High	Each active policy must generate a dedicated chat session (Chats record) between the customer and their assigned agent. Chat sessions are initialised via POST /api/chat/init with the policyId.
FR_COM_03	High	All messages must be persisted in the ChatMessages table with SenderId, SenderRole, Message text, Timestamp, and IsRead flag. Message history must be retrievable via GET /api/chat/{policyId} at any time.
FR_COM_04	Medium	The system must support read receipts. POST /api/chat/{policyId}/read must mark all unread messages in that conversation as IsRead = 1. Unread message counts must be returned via the notifications system.
FR_COM_05	High	The system must support WebRTC-based peer-to-peer video calling between customers and their assigned agents directly within the platform, without requiring external video conferencing tools.
FR_COM_06	High	Video call initiation requires the calling party to have an active, authenticated session. The WebRTC signaling exchange (SDP offer/answer and ICE candidates) must be facilitated through the platform's SignalR hub or a dedicated signaling service.
FR_COM_07	Medium	The frontend must implement proper WebRTC session teardown on component destruction (ngOnDestroy / useEffect cleanup) to release camera and microphone tracks and prevent hardware access from persisting after call termination.

5.5 Functional Requirements — Administration

Req. ID	Priority	Description
FR_ADM_01	Critical	Admin must be able to create Agent accounts via POST /api/admin/create-agent with: fullName, email, phoneNumber, bankAccountNumber (16-digit), and password. The system must store InitialPassword and dispatch login credentials automatically via email.
FR_ADM_02	Critical	Admin must be able to create Claims Officer accounts via POST /api/admin/create-claim-officer with the same fields as agent creation. Automated credential email dispatch applies equally.
FR_ADM_03	High	Admin must be able to view all platform users via GET /api/admin/all-users and delete any user via DELETE /api/admin/delete-user/{id}. Deletion must respect cascade rules defined in the database schema.

FR_ADM_04	Critical	GET /api/admin/admin-stats must return real-time KPI statistics: total user count by role, policy count by status (Pending/Active/Approved/Rejected/Expired), claim count by status, total claims amount requested, and total approved claims amount.
FR_ADM_05	High	The Admin dashboard must display real-time workload metrics per agent (current assigned policy count) via GET /api/admin/agents-with-load and per officer (current assigned claim count) via GET /api/claim/admin/officers to enable evidence-based assignment decisions.
FR_ADM_06	High	GET /api/report/unified-payments must return role-filtered financial reports: Admins see all payments and payouts; Agents see payments for their portfolio; Claims Officers see claim payout transactions they processed.

5.6 Functional Requirements — AI Integrations

Req. ID	Priority	Description
FR_AI_01	High	When a customer clicks 'Interest to Buy' on any policy, the frontend must open an AI-powered chatbot sidebar without navigating away from the policy page. The chatbot must be context-aware — it must know which specific policy the customer is viewing.
FR_AI_02	High	The policy chatbot must be powered by an N8N workflow automation engine via webhook integration from the AcciSure API. It must be capable of answering questions about coverage amounts, exclusions, tier differences, premium factors, and nominee requirements in natural language.
FR_AI_03	Critical	When a Claims Officer opens an assigned claim for review, the system must automatically trigger an N8N AI analysis webhook containing all ClaimDocument FileUrls. The AI workflow must return a structured JSON response within an acceptable response window.
FR_AI_04	Critical	The AI claims analysis response must include all four components: (1) document summary in plain language, (2) risk factor score categorised as Low / Medium / High, (3) suggested claim amount range in INR, (4) overall risk rating. Partial responses must be handled gracefully.
FR_AI_05	High	If the N8N AI service is unavailable or returns an error, the Claims Officer interface must still display the original claim details and documents. A clear error message must indicate that AI analysis is temporarily unavailable. The officer must be able to proceed with manual review.

6. API Reference

The AcciSure REST API is hosted at https://api.insuranceplatform.com and consists of 10 controllers exposing 44 endpoints. All protected endpoints require Authorization: Bearer {JWT_TOKEN}. The API uses application/json for all standard requests and multipart/form-data for file uploads. All monetary values are in INR. All datetimes use ISO 8601 UTC format.

6.1 AuthController — /api/auth

#	Method	Endpoint	Auth	Description
1.1	POST	/api/auth/register	None	Register customer with OTP + CAPTCHA verification. Returns JWT, roles, userId.
1.2	POST	/api/auth/login	None	Authenticate user. Returns JWT Bearer token, email, roles[], userId.

6.2 AdminController — /api/admin

All 11 endpoints require Admin role. Unauthorized access returns HTTP 403.

#	Method	Endpoint	Description
2.1	POST	/api/admin/create-agent	Create agent account. Auto-dispatches credentials via email.
2.2	POST	/api/admin/create-claim-officer	Create claims officer account. Auto-dispatches credentials via email.
2.3	GET	/api/admin/agents	List all agent accounts in the system.
2.4	GET	/api/admin/claim-officers	List all claims officer accounts.
2.5	DELETE	/api/admin/delete-user/{id}	Permanently delete any user. Cascade rules enforced.
2.6	GET	/api/admin/policy-requests	All policy applications across all statuses.
2.7	GET	/api/admin/agents-with-load	Agents with current assigned policy count for workload-based assignment.
2.8	POST	/api/admin/assign-agent	Assign policy application to agent. Body: { applicationId, agentId }.
2.9	GET	/api/admin/all-users	All platform users: customers, agents, and officers.
2.10	GET	/api/admin/all-claims	Master view of all insurance claims across the platform.
2.11	GET	/api/admin/admin-stats	KPI dashboard: user counts, policy/claim counts by status, total amounts.

6.3 AgentController — /api/agent

All endpoints require Agent role.

#	Method	Endpoint	Description
3.1	GET	/api/agent/my-requests	Fetch all policy applications assigned to the logged-in agent.
3.2	POST	/api/agent/review-request	Approve or reject application. Body: { applicationId, status: 'Approved'|'Rejected' }.
3.3	GET	/api/agent/my-customers	List all customers with active policies under this agent.
3.4	GET	/api/agent/commission-stats	Total commission earned and per-policy commission breakdown.
3.5	GET	/api/agent/analytics	Full analytics: coverage portfolio, tier breakdown, monthly commission trends, customer counts, best-performing category.

6.4 PolicyController — /api/policy

#	Method	Endpoint	Auth	Description
4.1	GET	/api/policy/configuration	Public	All policy plans, tiers, pricing, benefits, and age multipliers.
4.2	POST	/api/policy/calculate-premium	JWT	Premium quote without submitting. Applies all risk multipliers.
4.3	POST	/api/policy/apply	JWT	Submit formal application. Returns applicationId, calculatedPremium, status: Pending.
4.4	GET	/api/policy/my-policies	JWT	All policies for authenticated customer with coverage and status fields.

6.5 ClaimController — /api/claim

#	Method	Endpoint	Role	Description
5.1	POST	/api/claim/raise	Customer	File claim (multipart/form-data). Uploads docs to ImageKit. Max 10 files, 5MB each.
5.2	GET	/api/claim/my-claims	Customer	All claims raised by the authenticated customer with status and amounts.
5.3	GET	/api/claim/admin/pending	Admin	All pending claims awaiting officer assignment.
5.4	GET	/api/claim/admin/officers	Admin	Officer workload — current claim count per officer.
5.5	POST	/api/claim/admin/assign	Admin	Assign a claims officer to a pending claim.

5.6	GET	/api/claim/officer/my-requests	ClaimOfficer	All claims assigned to the logged-in officer (claims queue).
5.7	POST	/api/claim/officer/review	ClaimOfficer	Approve/reject claim. Body: { claimId, status, approvedAmount, remarks }.
5.8	GET	/api/claim/agent/customer-claims	Agent	Claims raised by the agent's portfolio customers.

6.6 ChatController — /api/chat

#	Method	Endpoint	Description
6.1	GET	/api/chat/list	All conversations for authenticated user with last-message preview.
6.2	GET	/api/chat/{policyId}	Full message history for a specific policy chat session.
6.3	POST	/api/chat/init	Initialise new chat session for a policy. Body: { policyId }.
6.4	POST	/api/chat/{policyId}/read	Mark all messages in policy chat as read (read receipts).

6.7 NotificationsController — /api/notifications

#	Method	Endpoint	Description
7.1	GET	/api/notifications	All notifications for authenticated user, ordered by CreatedAt descending.
7.2	GET	/api/notifications/unread-count	Returns integer count of unread notifications for badge display.
7.3	POST	/api/notifications/{id}/read	Mark specific notification as read by its GUID.
7.4	POST	/api/notifications/read-all	Mark all notifications as read in a single call.

6.8–6.10 Payment, Report, and Dashboard Controllers

#	Method	Endpoint	Role	Description
8.1	POST	/api/payment/process	Customer	Process premium payment. Returns policyStatus: Active.
9.1	GET	/api/report/unified-payments	Multi-Role	Role-filtered financial report for Admin, Agent, and ClaimOfficer.
10.1	GET	/api/dashboard/customer	Customer	Returns 200 OK if caller has Customer role; 403 otherwise.
10.2	GET	/api/dashboard/agent	Agent	Returns 200 OK if caller has Agent role; 403 otherwise.
10.3	GET	/api/dashboard/claim-officer	ClaimOfficer	Returns 200 OK if caller has ClaimOfficer role; 403 otherwise.
10.4	GET	/api/dashboard/admin	Admin	Returns 200 OK if caller has Admin role; 403 otherwise.

6.11 Standard Error Response Format

All API errors follow a consistent JSON response structure to enable predictable frontend error handling:

HTTP Code	Status	Scenario
200	OK	Request processed successfully
201	Created	Resource created successfully (registration, policy application, claim filing)
400	Bad Request	Validation failure or business rule violation (e.g., 'Email already exists', 'Age must be between 18 and 60')
401	Unauthorized	Missing, expired, or cryptographically invalid JWT token
403	Forbidden	Authenticated user lacks the required role for this endpoint
404	Not Found	Requested resource (user, policy, claim) does not exist
429	Too Many Requests	Rate limit exceeded — 100 requests/minute/user
500	Internal Server Error	Unhandled server-side exception — logged to Serilog for investigation

7. Non-Functional Requirements

7.1 Performance Requirements

Req. ID	Priority	Requirement
NFR_PERF_01	Critical	Admin dashboard aggregate statistics (GET /api/admin/admin-stats) executed against 50,000+ policy records must resolve within 200 milliseconds through indexed key queries that avoid full table scans.
NFR_PERF_02	High	Standard CRUD API endpoints (list, create, update, delete) must respond within 500 milliseconds under normal load conditions (up to 500 concurrent users).
NFR_PERF_03	High	SignalR chat messages must be delivered to recipients within 100 milliseconds on stable network connections, measured from server receipt to client rendering.
NFR_PERF_04	Medium	Document uploads to ImageKit CDN must complete within 10 seconds for files up to 5 MB on standard broadband connections (10 Mbps upload).
NFR_PERF_05	Medium	N8N AI analysis responses for claims must be returned within 30 seconds of trigger. If exceeded, the Claims Officer interface must display a timeout warning and allow manual review to proceed.
NFR_PERF_06	High	The frontend initial page load (LCP — Largest Contentful Paint) must complete within 3 seconds on a standard 4G mobile connection, achieved through lazy loading, code splitting, and Tailwind CSS purging.

7.2 Security Requirements

Req. ID	Priority	Requirement
NFR_SEC_01	Critical	All passwords must be hashed using bcrypt with a minimum cost factor of 10 via ASP.NET Core Identity. Plain-text passwords must never be stored, logged, or transmitted.
NFR_SEC_02	Critical	All JWT tokens must be signed with a strong secret key (minimum 256-bit entropy) using HMAC-SHA256. Token expiry must be enforced; expired tokens must be rejected on every protected request.
NFR_SEC_03	Critical	All API communications must occur over HTTPS (TLS 1.2+). HTTP connections must be redirected to HTTPS. HSTS headers must be configured on production deployments.
NFR_SEC_04	High	Rate limiting of 100 requests per minute per user/IP must be enforced at the API level using ASP.NET Core rate limiting middleware. Clients exceeding this limit receive HTTP 429 with a Retry-After header.
NFR_SEC_05	High	WebRTC video streams must use DTLS-SRTP encryption as mandated by the WebRTC specification. Unencrypted media streams must not be permitted.
NFR_SEC_06	High	All database queries must use EF Core parameterized queries. Raw SQL string concatenation is prohibited to prevent SQL injection attacks.
NFR_SEC_07	Medium	ImageKit-generated document URLs must be served over HTTPS. Sensitive claim documents should use signed URLs where supported by ImageKit's API to restrict unauthorized access.

7.3 Availability and Reliability Requirements

Req. ID	Priority	Requirement
NFR_AVAIL_01	Critical	The frontend, deployed via Vercel's serverless CDN infrastructure, must maintain uptime exceeding 99.9% monthly (approximately 8.7 hours maximum annual downtime).
NFR_AVAIL_02	Critical	The backend API must be containerised within fault-tolerant infrastructure with automatic restart on failure. Target uptime is 99.5% monthly (approximately 3.6 hours maximum monthly downtime).
NFR_AVAIL_03	High	The SQL Server database must be backed up daily with point-in-time recovery capability. Recovery Point Objective (RPO) must not exceed 24 hours. Recovery Time Objective (RTO) must not exceed 4 hours.
NFR_AVAIL_04	Medium	If the N8N AI service is unavailable, claims review must continue to function without AI analysis. Officers must receive a clear notification that AI assistance is temporarily offline rather than the entire interface failing.

7.4 Usability Requirements

Req. ID	Priority	Requirement
NFR_USE_01	High	The customer registration, policy application, and claim filing flows must each be completable by a first-time user within 10 minutes with no prior training, aided by inline field descriptions and validation error messages.
NFR_USE_02	High	All form validation error messages must appear inline adjacent to the relevant field immediately on blur or on submit attempt. Generic top-level error banners must not be the sole form of validation feedback.
NFR_USE_03	Medium	The platform must be fully responsive across screen widths from 320px (mobile) to 2560px (ultrawide desktop), meeting WCAG 2.1 Level AA accessibility standards for colour contrast and keyboard navigation.
NFR_USE_04	Medium	Toast notifications for status changes (policy approved, claim filed, message received) must appear within 2 seconds of the triggering event and auto-dismiss after 5 seconds unless interacted with.

7.5 Maintainability Requirements

Req. ID	Priority	Requirement
NFR_MAIN_01	High	All backend services must be registered with ASP.NET Core's built-in dependency injection container in Program.cs. No service instantiation via 'new' keyword in controllers is permitted.
NFR_MAIN_02	High	Structured logging must be implemented using Serilog with minimum Info level for all API operations. Error-level logs must include stack traces and request context. Logs must be retained for a minimum of 30 days.
NFR_MAIN_03	Medium	All API endpoints must be automatically documented via Swagger/OpenAPI (Swashbuckle) with accurate request/response models, authentication requirements, and error codes.

8. External Interface Requirements

8.1 User Interface Requirements

The AcciSure frontend must meet the following interface standards:

- Technology: Angular 21 (primary) or React.js with Vite/Next.js as SPA framework. Component-based architecture with strict TypeScript typing.
- Styling: Tailwind CSS 4.0 utility-first framework with PostCSS processing and automatic unused class purging. Custom design system colours must match corporate branding (Primary: #1B3A6B, Accent: #2563A8, Background: #F0F9F4).
- State Management: RxJS 7.8 reactive streams (Angular) or Redux Toolkit / React Context API (React) for global auth state, notifications, and real-time updates.
- Forms: Angular Reactive Forms (FormBuilder) or React Hook Form with Yup schema validation. All forms must implement inline validation with descriptive error messages.
- Data Visualization: Chart.js or Recharts for agent analytics dashboards and admin KPI charts. Charts must be responsive and support mobile viewports.
- Document Export: jsPDF and jspdf-autotable for invoice and policy PDF exports directly in the browser.
- Chat UI: Real-time messaging interface with conversation list, message bubbles with timestamps, unread count badges, and read receipt indicators.
- Video UI: Video calling interface with local and remote video streams in <video> elements, call accept/reject controls, mute/camera-off toggles, and full call teardown on component destruction.
- Notification Panel: Slide-out notification panel showing all system events with read/unread status and timestamps.

8.2 Software Interface Requirements

AcciSure integrates with the following external software systems:

System	Integration Method	Purpose and Details
N8N Workflow Engine	Outbound HTTP Webhook (POST)	AI policy chatbot responses and claims document analysis. N8N must be self-hosted. Webhook URLs configured in AcciSure API appsettings.json.
ImageKit.io	ImageKit .NET SDK	Claim document upload and CDN URL generation. API Key, Private Key, and URL Endpoint stored in secure configuration. SDK handles multipart upload and returns FileId and FileUrl.
Twilio SMS	Twilio .NET SDK / REST API	OTP dispatch via SMS during customer registration. Account SID and Auth Token stored in secure configuration. Messages sent from registered Twilio number.
Google reCAPTCHA	Frontend JS widget + backend token verification	reCAPTCHA v2 (checkbox) or v3 (invisible) integrated in registration form. Frontend generates token; backend verifies with Google Siteverify API using Site Secret Key.
MailKit / SendGrid	SMTP / SendGrid .NET SDK	Automated credential email dispatch to newly created Agent and ClaimOfficer accounts. HTML email templates with platform URL, login credentials, and onboarding instructions.

STUN/TURN Servers	WebRTC ICE Configuration	Session traversal for WebRTC video calls through NAT boundaries. Agora SDK, Jitsi, or Daily.co provides managed STUN/TURN infrastructure. ICE server configuration injected into RTCPeerConnection.
Microsoft SignalR	@microsoft/signalr client library	WebSocket-based real-time chat. HubConnectionBuilder connects via JWT Bearer token to /chatHub. Supports automatic reconnection, fallback to Long Polling if WebSockets unavailable.
SQL Server	EF Core 8 with SQL Server Provider	Primary relational data store. LINQ-based queries, parameterized SQL, automatic migration management. Connection string managed via environment-specific appsettings.

8.3 Hardware Interface Requirements

AcciSure is a web-based platform with no direct hardware dependencies beyond standard computing devices. The following hardware requirements apply:

Requirement	Details
Camera (Webcam)	Required for WebRTC video calling feature. Any device with a functional integrated or external webcam is supported. Access requested via browser getUserMedia API with graceful degradation if unavailable.
Microphone	Required for WebRTC audio in video calls. Access requested simultaneously with camera via getUserMedia. Audio-only calls with camera disabled are supported.
Internet Connectivity	Minimum 1 Mbps upload/download for standard API and chat usage. Minimum 2 Mbps bidirectional for WebRTC video calling. 4G mobile or broadband recommended.
Storage (Client)	No local storage required beyond browser cache. JWT tokens may be stored in HttpOnly cookies or browser memory. No file system access required.
Server Hardware	Backend API requires minimum 2 vCPU, 4 GB RAM for development. Production deployment on containerised infrastructure (Docker/Kubernetes) with auto-scaling. SQL Server requires minimum 4 GB RAM and SSD-backed storage.

9. Innovations and Unique Selling Propositions

AcciSure differentiates itself from conventional insurance platforms through nine implemented innovations and two planned upcoming features. Each innovation directly addresses a documented pain point in the traditional insurance lifecycle.

9.1 Implemented Innovations

Innovation 01 — AI Policy Chatbot via N8N

Problem	Insurance policies are complex. Customers historically called helplines or visited offices for basic questions, causing high drop-off rates at purchase.
Solution	When a customer clicks 'Interest to Buy', an N8N-powered AI chatbot sidebar opens inline — context-aware of the specific policy. It answers questions about coverage, exclusions, tiers, premiums, and nominees in natural language without page navigation.
Technology	N8N Workflow Automation Engine, ASP.NET Core webhook integration, Angular chatbot UI component
Business Impact	Reduces policy misunderstanding, increases purchase conversion, eliminates inbound support calls, scales self-service without headcount increase.

Innovation 02 — AI Claims Intelligence via N8N

Problem	Claims officers manually read through stacks of documents — FIR reports, hospital bills, medical certificates — with no AI assistance. Hours per claim, inconsistent decisions.
Solution	When an officer opens a claim, an N8N AI workflow automatically retrieves all document URLs, analyses content, and returns: (1) document summary, (2) risk score (Low/Medium/High), (3) suggested payout range, (4) risk rating. The officer retains final decision authority.
Technology	N8N AI workflow, ImageKit CDN document URLs, ASP.NET Core ClaimController webhook trigger, structured JSON response
Business Impact	Reduces review time from hours to minutes, improves decision consistency, enables higher claim volume without additional staff, reduces fraud risk.

Innovation 03 — In-Platform Video Calling (WebRTC)

Problem	Customer-agent video consultations required external tools (Zoom, Google Meet) and manual coordination, adding friction and removing context from the conversation.
Solution	Native peer-to-peer WebRTC video calling embedded within the platform. Customers initiate calls from the chat interface; agents join without leaving the platform. JWT session context is carried into the call.
Technology	WebRTC API, Agora SDK / Jitsi for STUN/TURN, SignalR for call signaling, DTLS-SRTP encryption
Business Impact	Eliminates external tool coordination overhead, strengthens customer-agent relationships, reduces claim disputes, increases customer retention.

Innovation 04 — Real-Time WhatsApp-Style Chat (SignalR)

Problem	Customer-agent communication was limited to asynchronous phone calls and emails — poorly documented, easily lost, and friction-heavy.
Solution	Full in-platform messaging using SignalR WebSockets. Every active policy generates a dedicated chat session. Messages are delivered instantly, stored persistently, and include read receipts — mirroring familiar messaging app UX.
Technology	ASP.NET Core SignalR hub, ChatMessages SQL table, React/Angular real-time event listeners, JWT-authenticated connections
Business Impact	Eliminates helpline calls for routine queries, creates auditable communication trail per policy, increases customer trust.

Innovation 05 — Smart Load-Based Assignment

Problem	Manual, arbitrary assignment of applications and claims created bottlenecks where some staff were overwhelmed while others remained idle.
Solution	Admin dashboard displays real-time workload metrics per agent and officer (current assignment counts). One-click assignment to the least-loaded staff member, with immediate notification dispatched on assignment.
Business Impact	Eliminates workload imbalances, prevents staff burnout, ensures consistent service quality, reduces average processing time.

Innovation 06 — Dynamic Coverage Balance Tracking

Problem	Customers had no real-time visibility into remaining coverage after claim approvals. Reconciliation required contacting the insurer and waiting for statements.
Solution	Upon every claim approval, the backend atomically deducts the approved amount from RemainingCoverageAmount and updates TotalApprovedClaimsAmount in a single EF Core transaction. Customers see current balances in real time via GET /api/policy/my-policies.
Business Impact	Builds trust through transparency, eliminates statement requests, prevents over-claiming, provides clean audit trail.

Innovation 07 — Cloud Document Management (ImageKit)

Problem	Local document storage created backup, accessibility, and scaling challenges. Email-based document transfer introduced security risks.
Solution	ImageKit.io SDK used for all claim document uploads. FileId and FileUrl stored in ClaimDocuments table. Documents served from ImageKit CDN globally. N8N AI workflow consumes URLs directly for analysis.
Business Impact	Eliminates storage maintenance overhead, ensures global document accessibility, enables AI analysis pipeline, scales without architectural changes.

Innovation 08 — OTP + CAPTCHA Dual Verification

Problem	Insurance platforms are high-value targets for fraudulent account creation. Single-factor registration is insufficient against bot-driven fake account creation.
Solution	Two-channel verification: CAPTCHA prevents bot submissions at form level; SMS OTP confirms phone number ownership before account creation. PhoneNumberConfirmed set to true only after OTP success.
Business Impact	Prevents fraudulent accounts, ensures verified phone contact for all customers, reduces downstream fraud risk, builds regulatory compliance into onboarding.

Innovation 09 — Automated Staff Credential Email Dispatch

Problem	Manual credential creation and insecure distribution via WhatsApp or verbal communication delayed staff onboarding and introduced security risk.
Solution	On Admin creation of any Agent or ClaimOfficer account, the system automatically dispatches a professionally formatted HTML email containing login credentials, platform URL, and first-login instructions via MailKit/SendGrid.
Business Impact	Staff operational within minutes of account creation, eliminates credential sharing security risks, consistent onboarding experience.

9.2 Upcoming Features

Feature	Status	Description
U1 — AI Voice Call Assistant	Planned — v2.0.0	Text-to-speech and speech-to-text AI policy assistant. Customers speak naturally to the AI chatbot using Web Speech API / Google TTS/STT. Proactively introduces policy details via voice. Full conversation transcript displayed simultaneously.
U2 — Multilingual Support	Planned — v2.0.0	Regional language support for Hindi, Telugu, Tamil, Kannada, Bengali, Marathi. Language preference stored per user. N8N AI chatbot responds in selected language. Google Translate API / DeepL integration as middleware translation layer.

9.3 Competitive Differentiation Summary

Capability	Conventional Platforms	AcciSure
Pre-purchase AI guidance	FAQ pages or helpline	✓ Contextual AI chatbot inline on policy page
Claims document analysis	Manual officer review	✓ AI-generated summary, risk score & payout range
Customer-agent video	External apps (Zoom, Meet)	✓ Native in-platform WebRTC video calling
Customer-agent messaging	Email or phone only	✓ Real-time WhatsApp-style chat with history
Staff workload assignment	Manual / arbitrary	✓ Live load metrics with one-click smart assignment

Coverage balance tracking	Monthly statements	✓ Real-time auto-deduction on every approval
Document storage	Email or local storage	✓ Cloud CDN (ImageKit) with AI-accessible URLs
Registration security	Email + password only	✓ Dual-gate: OTP verification + CAPTCHA
Staff credential delivery	Manual email or verbal	✓ Automated email on account creation

10. Workflow Summaries

10.1 Customer Workflow — Login to Logout

Step	Action	System Behaviour
1	**Registration**	Customer fills registration form, completes CAPTCHA, verifies phone via OTP. POST /api/auth/register creates account and returns JWT.
2	**Login**	POST /api/auth/login returns JWT. GET /api/dashboard/customer validates Customer role access. Customer lands on dashboard.
3	**Policy Exploration**	Customer browses GET /api/policy/configuration — all plans with tiers, pricing, and benefits displayed without authentication.
4	**AI Chatbot**	Customer clicks 'Interest to Buy'. N8N chatbot sidebar opens context-aware of the selected policy. Customer asks questions; AI responds. Customer closes when satisfied.
5	**Premium Calculation**	Customer fills applicant details. POST /api/policy/calculate-premium returns premium quote without committing to application.
6	**Policy Application**	Customer submits POST /api/policy/apply. PolicyApplication created with status 'Pending'. Notification dispatched to customer.
7	**Assignment & Review**	Admin assigns agent via POST /api/admin/assign-agent. Agent reviews and approves/rejects via POST /api/agent/review-request. Customer notified of outcome.
8	**Premium Payment**	Customer processes payment via POST /api/payment/process. Policy status becomes 'Active'. StartDate/ExpiryDate set. Agent receives 10% commission.
9	**Chat & Video**	Customer opens chat section. POST /api/chat/init creates session. Real-time SignalR messaging with assigned agent. Video call initiable from chat UI.
10	**Raise Claim**	Customer selects active policy, clicks 'Raise Claim'. Fills incident details, uploads documents. POST /api/claim/raise uploads to ImageKit; metadata stored in ClaimDocuments.
11	**Claim Progress**	Admin assigns officer. Officer reviews with AI analysis. Officer approves/rejects. RemainingCoverageAmount auto-updated. Customer notified. GET /api/claim/my-claims shows status.
12	**Logout**	JWT token cleared from client storage. SignalR connection terminated. Session ends.

10.2 Agent Workflow

Step	Action	System Behaviour
1	**Login**	Agent receives credentials via automated email from Admin. POST /api/auth/login. GET /api/dashboard/agent validates Agent role.
2	**View Applications**	GET /api/agent/my-requests returns all policy applications assigned by Admin with full applicant details.
3	**Review & Decide**	Agent reads ApplicationDataJson (applicant info, nominee, family members). POST /api/agent/review-request with status 'Approved' or 'Rejected'. Customer notified.
4	**Customer Communication**	GET /api/chat/list shows all customer conversations. Real-time messaging and video calling with assigned customers.
5	**Analytics**	GET /api/agent/analytics for full performance analytics. GET /api/agent/commission-stats for earnings. GET /api/report/unified-payments for financial report.

10.3 Claims Officer Workflow

Step	Action	System Behaviour
1	**Login**	Officer receives credentials via automated email. POST /api/auth/login. GET /api/dashboard/claim-officer validates ClaimOfficer role.
2	**View Claims Queue**	GET /api/claim/officer/my-requests retrieves all claims assigned to this officer with current status.
3	**Select Claim**	Officer clicks 'View Details'. Full incident info, hospital details, and ImageKit document URLs displayed.
4	**AI-Assisted Review**	N8N AI workflow triggered automatically. Returns: document summary, risk factor, suggested payout range, risk rating. Displayed alongside claim details.
5	**Decision**	POST /api/claim/officer/review with claimId, status, approvedAmount, remarks. If approved: RemainingCoverageAmount atomically deducted, TotalApprovedClaimsAmount updated, customer notified.

10.4 Admin Workflow

Step	Action	System Behaviour
1	**Login**	Admin logs in with pre-configured credentials. GET /api/dashboard/admin validates Admin role.
2	**Dashboard Overview**	GET /api/admin/admin-stats returns platform-wide KPIs: user counts, policy/claim counts by status, total and approved claim amounts.
3	**Create Staff**	POST /api/admin/create-agent or /create-claim-officer. System stores InitialPassword, dispatches credential email automatically. Staff operational immediately.

4	**Assign Applications**	GET /api/admin/agents-with-load shows agent workloads. POST /api/admin/assign-agent assigns selected application. Agent notified.
5	**Assign Claims**	GET /api/claim/admin/officers shows officer workloads. POST /api/claim/admin/assign assigns pending claim to selected officer. Officer notified.
6	**Financial Oversight**	GET /api/report/unified-payments provides complete platform-wide financial report. GET /api/admin/admin-stats for ongoing KPI monitoring.

11. Appendices

Appendix A — Policy Tier Summary

Plan	Coverage	Price/yr	Type	Key Benefits
Silver	₹5 Lakhs	₹3,000+	Individual	Accidental Death, Permanent Disability, Hospitalisation Support
Gold ★	₹10 Lakhs	₹5,000+	Individual	Accidental Death, Permanent Disability, Partial Disability Cover
Premium	₹20 Lakhs	₹8,000+	Individual	Worldwide Emergency, ICU Cover, Full Disability Cover
Family Silver	₹10 Lakhs	₹7,000+	Family (4 members)	Family Accidental, Hospitalisation, Children Education Support
Family Gold ★	₹20 Lakhs	₹12,000+	Family (4 members)	Family Accidental, ICU, Children Education, Ambulance Cover
Family Premium	₹40 Lakhs	₹18,000+	Family (4 members)	Worldwide Emergency, Full Disability, ICU + Surgery Cover

Appendix B — Premium Calculation Risk Multipliers

Factor	Value	Multiplier Applied
Age 18–30	Low Risk	1.0x — Base premium rate applied
Age 31–45	Medium Risk	1.2x — 20% loading on base premium
Age 46–60	High Risk	1.5x — 50% loading on base premium
Profession	Variable	High-risk professions (mining, construction) attract higher multipliers
Smoking Habit	Yes / No	Smoking incurs additional risk loading on calculated premium
Alcohol Habit	Yes / No	Regular alcohol consumption incurs additional risk loading

Travel (km/month)	Continuous	Higher monthly travel distance increases premium proportionally

Appendix C — Database Relationship Summary

Table	Foreign Keys	Delete Rule
AspNetUserRoles	UserId → AspNetUsers, RoleId → AspNetRoles	CASCADE on both
AspNetUserClaims	UserId → AspNetUsers	CASCADE
Notifications	UserId → AspNetUsers	CASCADE
PolicyApplications	UserId → AspNetUsers (CASCADE), AssignedAgentId → AspNetUsers	Mixed — CASCADE on owner, NO ACTION on agent
InsuranceClaims	PolicyApplicationId → PolicyApplications, UserId → AspNetUsers, AssignedClaimOfficerId → AspNetUsers	Mixed — CASCADE on owner, NO ACTION on others
ClaimDocuments	ClaimId → InsuranceClaims	CASCADE
Chats	PolicyId → PolicyApplications	CASCADE
ChatMessages	ChatId → Chats	CASCADE

Appendix D — Technology Decision Rationale

Technology	Why Chosen	Alternatives Considered
ASP.NET Core 8	Native SignalR, first-class JWT/Identity, strong EF Core ORM, high performance	Node.js/Express, Django — lack native SignalR and Identity
SQL Server + EF Core	Strong relational integrity for financial data, LINQ type-safe queries, migration management	PostgreSQL, MongoDB — less mature EF integration; NoSQL unsuitable for financial data
SignalR	First-class ASP.NET integration, automatic fallback (WebSocket→Long Polling→SSE)	Socket.IO — Node-centric; raw WebSockets — require manual protocol management
N8N	Visual AI workflow, webhook-based REST integration, self-hostable for data privacy	Zapier — not self-hostable; LangChain — higher backend development cost
ImageKit.io	Purpose-built media CDN with SDK, automatic optimisation, secure URLs	AWS S3 — higher setup; Cloudinary — media-focused; local storage — not scalable
JWT Bearer Auth	Stateless, scalable, role-claims built-in, standard across mobile and web clients	Session-based auth — not stateless; OAuth only — overkill for controlled-role platform

12. References

12.1 Internal Documents

- AcciSure Project Documentation Report — Complete Technical & Functional Reference
- AcciSure Innovation & USP Documentation Report, March 2026
- AcciSure Frontend Developer Guide & Technical Documentation (FRONTEND_README.md)
- AcciSure Backend API Documentation (BACKEND_README.md)

12.2 Technical Standards and Specifications

- IEEE 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- ISO/IEC 25010:2011 — Systems and software quality models
- WCAG 2.1 Level AA — Web Content Accessibility Guidelines, W3C, 2018
- RFC 7519 — JSON Web Token (JWT) Standard, IETF, 2015
- RFC 8825 — WebRTC: An Introduction and Requirements, IETF, 2021
- OWASP Top 10 2021 — Open Web Application Security Project, 2021

12.3 Framework and Platform Documentation

- Microsoft ASP.NET Core 8.0 Documentation — https://learn.microsoft.com/en-us/aspnet/core/
- Microsoft ASP.NET Core SignalR Documentation — https://learn.microsoft.com/en-us/aspnet/core/signalr/
- Entity Framework Core 8 Documentation — https://learn.microsoft.com/en-us/ef/core/
- Angular 21 Documentation — https://angular.dev/
- Tailwind CSS 4.0 Documentation — https://tailwindcss.com/docs/
- WebRTC API — MDN Web Docs — https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API
- ImageKit.io Documentation — https://imagekit.io/docs/
- N8N Workflow Automation Documentation — https://docs.n8n.io/
- Google reCAPTCHA v3 Documentation — https://developers.google.com/recaptcha/docs/v3
- Twilio Programmable SMS Documentation — https://www.twilio.com/docs/sms
- SendGrid Email API Documentation — https://docs.sendgrid.com/